MANUAL DE ESPECIALIZACIÓN TÉCNICA

Desarrollo Web Full Stack, Automatización y Servidores Locales

Guía metodológica, arquitectónica y operativa integral para instructores y participantes. Desarrollada bajo estrictos estándares de ingeniería de software defensiva e infraestructura industrial. Incorpora marcos teóricos rigurosos, diagramas de flujo de datos, analogías didácticas, código fuente auditado línea por línea, protocolos de mitigación de vulnerabilidades OWASP Top 10, optimización de servidores locales multi-proceso y la implementación de un ecosistema monorepo distribuido de alta disponibilidad.

Módulo	Marco Conceptual, Hitos Técnicos y Competencias	Stack y Runtimes
Módulo 0	Reproducibilidad, Principio de Paridad Dev/Prod, Lockfiles y Quality Gates	Ubuntu 24.04 / WSL2
Módulo 1 (I)	Grafo Acíclico Dirigido (DAG), Curvas Ed25519 vs RSA, Git Hooks Client/Server	Git 2.47+ / Bash Hooks
Módulo 1 (II)	Parsing DOM/CSSOM, Layout Tree, Mobile-First, Tokens y Accesibilidad WCAG	HTML5 / CSS Grid / BEM
Módulo 2	Event Loop V8, Call Stack, Microtasks Queue vs Macrotasks, XSS y DocumentFragment	ES Modules / Fetch API
Módulo 3	Tokens JWT RFC 7519, Hashing Bcrypt con Cost Factor, Timing Attacks y Helmet	Node 24 LTS / Express 5
Módulo 4 (I)	Arquitectura MVC, Active Record, Inyecciones Mass Assignment y FormRequests	PHP 8.4 / Laravel 12
Módulo 4 (II)	Concurrencia ASGI vs WSGI, evasión del GIL con Uvicorn y GUI Thread-Safety	Python 3.13 / FastAPI / Tkinter
Módulo 5	Orquestación Monorepo Distribuida, Auditoría Técnica y Pruebas de Estrés	ApacheBench / Dev-All

Nombre del Participante:

Empresa / Planta:

Cédula / Documento ID:

Instructor Responsable:

MÓDULO 0: REPRODUCIBILIDAD Y ARQUITECTURA DEL ENTORNO BASE

ENTORNO BASE

0.1 Marco Teórico: Reproducibilidad, Lockfiles y Calidad de Entorno

El Principio de Paridad Dev/Prod: La expresión "en mi máquina sí funciona" revela una ausencia crítica de aislamiento de configuración. La reproducibilidad garantiza que cualquier máquina de desarrollo, servidor de pruebas o contenedor de integración continua genere resultados binarios idénticos. Exige 4 pilares inmutables: Sistema Operativo Base unificado (Ubuntu 24.04 LTS), Runtimes con versiones bloqueadas por política de equipo, dependencias congeladas mediante lockfiles con verificación de hashes criptográficos SHA-512, y configuración totalmente desacoplada en variables de entorno `.env`.

Impacto Real en Producción: Si dos desarrolladores ejecutan `npm install` sin un `package-lock.json` estricto, podrían obtener versiones menores diferentes con cambios incompatibles en librerías criptográficas, provocando que los tokens de sesión firmados en un nodo no puedan ser descifrados por otro en la red local.

Módulo	Runtime Requerido	Gestor de Paquetes	Lockfile / Checksum Criptográfico
1 - 2	Node.js 20/24 LTS	npm 10.x / 11.x	`package-lock.json` (integridad SHA-512)
3	Node.js 20/24 LTS	npm 10.x / 11.x	`package-lock.json` (versiones fijadas)
4 - I	PHP 8.2 / 8.4	Composer 2.x	`composer.lock` (hash content-hash)
4 - II	Python 3.11 / 3.13	pip + venv	`requirements.txt` (pinning ==)
5	Multi-runtime	npm / composer / pip	Lockfiles consolidados en Monorepo

0.2 Script de Instalación Base (Ubuntu 22.04 / 24.04 / WSL2)

BASH: INSTALACIÓN AUTOMATIZADA DE HERRAMIENTAS BASE — setup-base.sh

```bash
#!/bin/bash
set -e

echo "=== [1/6] Actualización del sistema base ==="
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential curl wget git zip unzip \
  software-properties-common ca-certificates gnupg lsb-release

echo "=== [2/6] Instalación de Node.js 20 LTS vía NodeSource ==="
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

echo "=== [3/6] Instalación de Python 3.11 + venv + Tkinter ==="
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install -y python3.11 python3.11-venv python3.11-dev python3.11-tk python3-pip
sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.11 1

echo "=== [4/6] Instalación de PHP 8.2 + Extensiones + Composer ==="
sudo add-apt-repository ppa:ondrej/php -y
sudo apt update
sudo apt install -y php8.2 php8.2-cli php8.2-common php8.2-curl \
  php8.2-mbstring php8.2-xml php8.2-zip php8.2-mysql php8.2-sqlite3 \
  php8.2-bcmath php8.2-intl
curl -sS https://getcomposer.org/installer | php
sudo mv composer.phar /usr/local/bin/composer
sudo chmod +x /usr/local/bin/composer

echo "=== [5/6] Instalación de SQLite y utilidades de red ==="
sudo apt install -y sqlite3 libsqlite3-dev apache2-utils jq

echo "=== [6/6] Verificación preliminar de versiones ==="
node -v && npm -v && python3 --version && php -v | head -1 && composer --version | head -1
```

0.3 Quality Gates y Auditoría de Sistema con Scripts Bash

¿Qué es un Quality Gate? Es una condición booleana estricta que detiene el flujo de trabajo si no se cumplen los requisitos mínimos de compatibilidad, herramientas o versiones. El script `verify-env.sh` actúa como la primera compuerta de validación: analiza el sistema mediante expresiones regulares POSIX y retorna código de salida 0 (éxito) o 1 (aborto inmediato), integrándose nativamente en terminales o pipelines de integración continua.

BASH: SCRIPT DE CONTROL PREVIO (scripts/verify-env.sh) — chmod +x scripts/verify-env.sh

```bash
#!/bin/bash
# scripts/verify-env.sh
set -eo pipefail

RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'; NC='\033[0m'
ERRORS=0

verificar_comando() {
  local cmd=$1
  local version_cmd=$2
  local expected=$3

  echo -n "Comprobando $cmd... "
  if ! command -v "$cmd" &> /dev/null; then
    echo -e "${RED}[NO ENCONTRADO]${NC}"
    ERRORS=$((ERRORS + 1))
    return
  fi

  local version_output
  version_output=$(eval "$version_cmd" 2>&1 | head -n 1)
  if echo "$version_output" | grep -qE "$expected"; then
    echo -e "${GREEN}[OK]${NC} ($version_output)"
  else
    echo -e "${YELLOW}[VERSION DISCREPANTE]${NC} Obtenido: '$version_output' (Esperado: '$expected')"
    ERRORS=$((ERRORS + 1))
  fi
}

echo "================================================"
echo "    AUDITORÍA DE ENTORNO DE DESARROLLO         "
echo "================================================"

verificar_comando "git" "git --version" "2\.[0-9]+"
verificar_comando "node" "node -v" "v20\."
verificar_comando "npm" "npm -v" "10\."
verificar_comando "python3" "python3 --version" "3\.1[1-9]"
verificar_comando "php" "php -v" "PHP 8\.2"
verificar_comando "composer" "composer --version" "Composer version 2\."
verificar_comando "sqlite3" "sqlite3 --version" "3\."

echo "------------------------------------------------"
if [ "$ERRORS" -eq 0 ]; then
  echo -e "${GREEN}✔ Entorno 100% verificado y listo para desarrollo.${NC}"
  exit 0
else
  echo -e "${RED}✖ Se encontraron $ERRORS errores en el entorno. Revisa la instalación.${NC}"
  exit 1
fi
```

Laboratorio 0.1 — Calibración del Sistema: Clona el repositorio, asigna permisos mediante `chmod +x scripts/verify-env.sh` y ejecuta `./scripts/verify-env.sh`. Comprueba que el código de retorno en la shell sea 0 ejecutando `echo $?`.

MÓDULO 1 (PARTE I): GIT INTERNALS, SSH ED25519 Y SERVIDORES LOCALES

GIT ARCHITECTURE

1.1 Teoría: El Grafo Acíclico Dirigido (DAG) y los 4 Objetos de Git

Estructura Interna de Git: Git no almacena diferencias (diffs) entre versiones; almacena instantáneas completas organizadas como un Grafo Acíclico Dirigido (DAG). Cada elemento reside en `.git/objects/` indexado por el hash SHA-1 (40 caracteres hexadecimales) de su contenido:
• **Blob (Binary Large Object):** Datos crudos de un archivo. No tiene nombre, ni fecha, ni permisos de archivo.
• **Tree:** Modela un directorio. Vincula nombres, permisos POSIX y hashes SHA-1 a blobs u otros sub-trees.
• **Commit:** Registro inmutable que apunta a un Tree raíz, contiene autor, committer, timestamp, mensaje y el hash del commit padre.
• **Tag Anotado:** Referencia fija sellada con mensaje, autor y firma criptográfica GPG opcional.

Criptografía SSH: Curva de Edwards Ed25519 vs. RSA: RSA depende de la factorización de números primos grandes (2048/4096 bits), lo cual consume mayor cómputo y genera firmas extensas. Ed25519 se fundamenta en la curva de Edwards Twisted 25519. Ofrece un nivel de seguridad superior con llaves públicas compactas de apenas 68 caracteres, previene ataques de canal lateral (timing attacks en el handshake SSH) y ejecuta operaciones de firma a muy alta velocidad.

1.2 Estructura del Proyecto y Reglas de .gitignore

TEXT: REGLAS ESTRICTAS DE EXCLUSIÓN (.gitignore) — .gitignore

```
node_modules/
vendor/
.env
.env.*
!.env.example
*.log
.DS_Store
Thumbs.db
dist/
build/
__pycache__/
*.pyc
.venv/
database/*.sqlite
```

1.3 Configuración Criptográfica y Llaves SSH Ed25519

BASH: GENERACIÓN DE LLAVES Y CONFIGURACIÓN POSIX — ssh-setup.sh

```bash
# 1. Configuración de identidad
git config --global user.name "Tu Nombre"
git config --global user.email "tu@onetel.cloud"
git config --global init.defaultBranch main

# 2. Generación del par de claves Ed25519 sin passphrase
ssh-keygen -t ed25519 -C "tu@onetel.cloud" -f ~/.ssh/id_ed25519_local -N ""

# 3. Permisos POSIX estrictos (Obligatorio en OpenSSH)
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_ed25519_local
chmod 644 ~/.ssh/id_ed25519_local.pub

# 4. Carga en el agente de autenticación
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519_local

# 5. Comprobación de autenticación con GitHub
ssh -T git@github.com || true
```

1.4 Git Hooks: Seguridad Perimetral y Despliegue Automatizado

Mecanismo de Disparo de Hooks: Git posee un sistema de interceptación nativo en `.git/hooks/`. Los scripts *Client-Side* (como `pre-commit`) corren localmente en la máquina del desarrollador para auditar sintaxis, formateo o fugas de secretos antes de crear el objeto commit. Los scripts *Server-Side* (como `post-receive`) corren en el servidor remoto tras recibir un push exitoso, permitiendo desplegar automáticamente el código sin requerir herramientas externas pesadas.

BASH: AUDITORÍA PREVENTIVA (.git/hooks/pre-commit) — `chmod +x .git/hooks/pre-commit`

```bash
#!/bin/bash
# .git/hooks/pre-commit
set -eo pipefail

echo "[PRE-COMMIT] Auditando integridad de la confirmación..."

# 1. Bloquear archivos .env accidentales en staging
ARCHIVOS_ENV=$(git diff --cached --name-only | grep -E '(^|/)\.env($|\.)' || true)
if [ -n "$ARCHIVOS_ENV" ]; then
  echo "============================================================"
  echo " [BLOQUEO DE SEGURIDAD] Archivo de entorno detectado:"
  echo " $ARCHIVOS_ENV"
  echo " Retíralo de staging con: git reset HEAD <archivo>"
  echo " Asegúrate de que figure en .gitignore."
  echo "============================================================"
  exit 1
fi

# 2. Bloquear archivos que superen 1 MB (1048576 bytes)
while IFS= read -r file; do
  [ -f "$file" ] || continue
  size=$(stat -c%s "$file" 2>/dev/null || stat -f%z "$file" 2>/dev/null)
  if [ "$size" -gt 1048576 ]; then
    echo "[BLOQUEO] Archivo demasiado grande (>1MB): $file ($size bytes)"
    exit 1
  fi
done < <(git diff --cached --name-only --diff-filter=ACM)

# 3. Detección heurística de tokens de API añadidos (+ en el diff)
DIFF_AGREGADO=$(git diff --cached -U0 | grep -E '^\+[^\+]' || true)
PATRONES_SECRETOS='(sk-[a-zA-Z0-9]{20,}|ghp_[a-zA-Z0-9]{30,}|AKIA[0-9A-Z]{16}|bearer\s+[a-zA-Z0-9\.\-_]{30,})'

if echo "$DIFF_AGREGADO" | grep -qiE "$PATRONES_SECRETOS"; then
  echo "[BLOQUEO] Posible token o clave privada detectada en el diff."
  exit 1
fi

echo "[PRE-COMMIT] ✔ Todas las validaciones de seguridad fueron superadas."
exit 0
```

BASH: CONFIGURACIÓN SERVIDOR BARE (scripts/setup-server.sh) — `post-receive`

```bash
#!/bin/bash
# scripts/setup-server.sh
set -eo pipefail

REPO_PATH="/var/repo/app.git"
TARGET="/var/www/app"

sudo mkdir -p "$REPO_PATH" "$TARGET"
sudo chown -R "$USER:$USER" "$REPO_PATH" "$TARGET"

if [ ! -d "$REPO_PATH/refs" ]; then
  git init --bare "$REPO_PATH"
fi

cat > "$REPO_PATH/hooks/post-receive" << 'HOOK_EOF'
#!/bin/bash
TARGET="/var/www/app"
GIT_DIR="/var/repo/app.git"
unset GIT_INDEX_FILE

while read -r oldrev newrev ref; do
  RAMA=$(git rev-parse --symbolic --abbrev-ref "$ref" 2>/dev/null || echo "$ref" | sed 's#refs/heads/##')
  if [ "$RAMA" = "main" ]; then
    echo "[POST-RECEIVE] Desplegando rama main en $TARGET..."
    mkdir -p "$TARGET"
    git --work-tree="$TARGET" --git-dir="$GIT_DIR" checkout -f main
  fi
done
```

MÓDULO 1 (PARTE II): HTML5 SEMÁNTICO, ACCESIBILIDAD Y RENDERING ENGINE

ACCESSIBLE DOM

2.1 El Pipeline de Renderizado del Navegador y Accesibilidad (WCAG 2.1)

De Bytes a Pantalla: Los motores como Blink o Gecko no pintan HTML directamente. Procesan los bytes crudos y construyen el *DOM Tree* (estructura jerárquica) y el *CSSOM Tree* (hoja de estilos computada). Ambos convergen en el *Render Tree*, que calcula la geometría exacta en la etapa de *Layout* (o Reflow) antes de realizar el *Paint* sobre la GPU. Usar etiquetas semánticas como `<header>`, `<main>` y `<article>` construye paralelamente el *Accessibility Tree* (AOM), permitiendo a lectores de pantalla y motores de búsqueda entender la arquitectura de la información sin relying en `<div>` genéricos.

HTML5: ESTRUCTURA SEMÁNTICA CON ARIA Y WCAG 2.1 (index.html) — dashboard-telemetria/index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Dashboard de telemetría de infraestructura">
  <title>Telemetría — Onetel Cloud</title>
  <link rel="stylesheet" href="css/main.css">
</head>
<body class="app">
  <a class="skip-link" href="#main">Saltar al contenido principal</a>

  <header class="app-header">
    <div class="brand">
      <span class="brand__icon" aria-hidden="true">◉</span>
      <h1 class="brand__title">Onetel Local Monitor</h1>
    </div>
    <nav class="nav" aria-label="Navegación principal">
      <ul class="nav__list">
        <li class="nav__item"><a class="nav__link nav__link--active" href="#">Panel</a></li>
        <li class="nav__item"><a class="nav__link" href="#servidores">Servidores</a></li>
        <li class="nav__item"><a class="nav__link" href="#metricas">Telemetría</a></li>
      </ul>
    </nav>
  </header>

  <main id="main" class="app-main">
    <section class="summary" aria-label="Resumen de indicadores clave">
      <article class="kpi">
        <h2 class="kpi__label">Servidores Activos</h2>
        <p class="kpi__value" id="kpi-activos">—</p>
      </article>
      <article class="kpi">
        <h2 class="kpi__label">CPU Promedio</h2>
        <p class="kpi__value" id="kpi-cpu">—</p>
      </article>
      <article class="kpi">
        <h2 class="kpi__label">RAM Libre Global</h2>
        <p class="kpi__value" id="kpi-ram">—</p>
      </article>
    </section>

    <section id="servidores" class="servers">
      <header class="servers__header">
        <h2 class="servers__title">Nodos Monitoreados</h2>
        <input class="servers__search" id="input-buscador" type="search" placeholder="Filtrar por hostname...">
      </header>
      <div class="servers__grid" id="contenedor-servidores" aria-live="polite"></div>
    </section>
  </main>

  <footer class="app-footer">
    <p>&copy; 2026 Onetel C.A. — División de Infraestructura y Servidores Locales.</p>
  </footer>
  <script type="module" src="js/app.js"></script>
</body>
</html>
```

2.2 Metodología Mobile-First y Tokens de Diseño

¿Por qué Mobile-First? Al diseñar desde pantallas reducidas (320px) hacia monitores de escritorio (1400px), el navegador carga de inmediato estilos lineales simplificados sin tener que procesar complejas sobreescrituras en dispositivos móviles. Los *Design Tokens* (variables CSS con `--`) centralizan colores, espaciados y tipografías en un solo lugar, permitiendo cambiar el tema corporativo de una aplicación completa modificando únicamente `:root`.

CSS: DESIGN TOKENS Y RESET DE CAJA (css/tokens.css & css/reset.css) — css/tokens.css & css/reset.css

```css
/* css/tokens.css */
:root {
  --color-primary: #0f172a;
  --color-accent: #2563eb;
  --color-bg: #f8fafc;
  --color-surface: #ffffff;
  --color-text: #0f172a;
  --color-text-muted: #64748b;
  --color-border: #e2e8f0;

  --color-success: #16a34a;
  --color-success-bg: #dcfce7;
  --color-danger: #dc2626;
  --color-danger-bg: #fee2e2;

  --font-body: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  --font-mono: ui-monospace, "Cascadia Code", "Fira Code", monospace;

  --space-2: 0.5rem;
  --space-4: 1rem;
  --space-6: 1.5rem;
  --radius-md: 10px;
}

/* css/reset.css */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  min-height: 100vh;
  line-height: 1.5;
  font-family: var(--font-body);
}
```

CSS: ESTRUCTURA DE LAYOUT CON MEDIA QUERIES (css/layout.css) — css/layout.css

```css
body {
  background-color: var(--color-bg);
  color: var(--color-text);
}

.app {
  min-height: 100vh;
  display: grid;
  grid-template-rows: auto 1fr auto;
}

.app-main {
  padding: 1rem;
  display: grid;
  gap: 1.5rem;
  max-width: 1400px;
  margin: 0 auto;
  width: 100%;
}

.summary {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}

.servers__grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}
```

2.3 Metodología BEM (Block, Element, Modifier)

Especificidad Plana y Mantenibilidad: Las colisiones de nombres y las cascadas infinitas son el principal problema de CSS en equipos grandes. BEM propone:
• **Bloque (`.card-server`):** Entidad independiente reutilizable.
• **Elemento (`.card-server__title`):** Parte estructural que no tiene sentido fuera del bloque.
• **Modificador (`.card-server--online`):** Estado o variante visual. Mantiene la especificidad baja (una sola clase) evitando el uso destructivo de `!important`.

CSS: COMPONENTES MODULARES BEM (css/components.css) — css/components.css

```css
/* Tarjetas KPI */
.kpi {
  background: var(--color-surface);
  padding: 1rem;
  border-radius: var(--radius-md);
  border: 1px solid var(--color-border);
}

.kpi__label {
  font-size: 0.875rem;
  color: var(--color-text-muted);
  font-weight: 500;
}

.kpi__value {
  font-size: 1.875rem;
  font-weight: 700;
  color: var(--color-accent);
  margin-top: 0.25rem;
}

/* Tarjetas de Servidores */
.card-server {
  background: var(--color-surface);
  padding: 1rem;
  border-radius: var(--radius-md);
  border: 1px solid var(--color-border);
  border-left: 5px solid var(--color-text-muted);
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}

.card-server--online {
  border-left-color: var(--color-success);
}

.card-server--offline {
  border-left-color: var(--color-danger);
}

.card-server__header {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.card-server__title {
  font-size: 1.125rem;
  font-weight: 600;
}

.badge {
  padding: 0.25rem 0.5rem;
  border-radius: 9999px;
  font-size: 0.75rem;
  font-weight: 700;
}

.badge--online {
  background: var(--color-success-bg);
  color: var(--color-success);
}

.badge--offline {
  background: var(--color-danger-bg);
  color: var(--color-danger);
```

MÓDULO 2: JAVASCRIPT MODERNO (ES6+), EVENT LOOP Y ASINCRONÍA

EVENT LOOP V8

3.1 La Mecánica Interna del Event Loop de V8

Pila de Ejecución y Colas de Tareas: JavaScript ejecuta sobre un solo hilo (*single thread*). Para no congelar la pantalla durante peticiones de red pesadas, el motor V8 delega el trabajo al entorno del navegador (Web APIs) y orquesta la ejecución mediante 3 estructuras:
1. **Call Stack:** Pila sincrónica donde se ejecutan las funciones activas LIFO (Last In, First Out).
2. **Microtasks Queue (Prioridad Absoluta):** Promesas (`.then()`, `await`, `queueMicrotask`). El Event Loop vacía COMPLETAMENTE esta cola antes de ceder el turno al renderizado de la UI.
3. **Macrotasks Queue:** `setTimeout`, `setInterval` y eventos de periféricos. Solo se procesa una por ciclo.

3.2 Capa de Red con Reintento Exponencial: js/api.js

JAVASCRIPT: CLIENTE RESILIENTE CON ABORTCONTROLLER — js/api.js

```javascript
// js/api.js
const SERVIDORES_API = window.API_SERVIDORES_URL || 'http://127.0.0.1:8000/api/servidores';
const METRICAS_API   = window.API_METRICAS_URL   || 'http://127.0.0.1:8001/api/v1/system/metrics';

/**
 * Petición con reintento exponencial y AbortController
 */
export async function fetchMetricas({ timeoutMs = 5000, intentos = 3 } = {}) {
  for (let i = 0; i < intentos; i++) {
    const controller = new AbortController();
    const timer = setTimeout(() => controller.abort(), timeoutMs);

    try {
      const res = await fetch(METRICAS_API, { signal: controller.signal });
      clearTimeout(timer);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return await res.json();
    } catch (err) {
      clearTimeout(timer);
      if (i === intentos - 1) throw err;
      await new Promise(r => setTimeout(r, 1000 * Math.pow(2, i)));
    }
  }
}

export async function fetchServidores(token = null) {
  const headers = { 'Accept': 'application/json' };
  if (token) headers['Authorization'] = `Bearer ${token}`;

  const res = await fetch(SERVIDORES_API, { headers });
  if (!res.ok) throw new Error(`HTTP ${res.status} al consultar servidores`);
  const body = await res.json();
  return Array.isArray(body) ? body : (body.data || []);
}
```

3.3 Estado Centralizado: js/state.js

JAVASCRIPT: GESTIÓN DE ESTADO REACTIVO LOCAL — js/state.js

```javascript
export const estadoApp = {
  servidores: [],
  metricas: null,
  filtroTexto: '',
  cargando: false,
  error: null
};

export function actualizarEstado(patch) {
  Object.assign(estadoApp, patch);
}
```

3.4 Reflow, Layout Thrashing y Vulnerabilidades XSS

¿Por qué DocumentFragment? Cada vez que ejecutas `parent.appendChild(node)` directamente en el DOM, el motor del navegador puede desencadenar un *Reflow* (recalculo de coordenadas de toda la página) y un *Repaint*. Si insertas 100 servidores en un bucle, provocarás 100 reflows consecutivos (*Layout Thrashing*). Al usar `DocumentFragment`, agrupas todos los nodos en un contenedor fantasma en memoria y realizas **una sola mutación real** en el DOM.

JAVASCRIPT: RENDER SEGURO Y EFICIENTE (js/render.js) — js/render.js

```
const contenedor = document.getElementById('contenedor-servidores');
const kpiActivos = document.getElementById('kpi-activos');
const kpiCpu = document.getElementById('kpi-cpu');
const kpiRam = document.getElementById('kpi-ram');

export function escapeHtml(str) {
  if (str === null || str === undefined) return '';
  const div = document.createElement('div');
  div.textContent = String(str);
  return div.innerHTML;
}

export function renderServidores(servidores, filtroTexto = '') {
  if (!contenedor) return;
  contenedor.innerHTML = '';

  const filtrados = servidores.filter(srv => {
    const nombre = srv.hostname || srv.nombre || '';
    return nombre.toLowerCase().includes(filtroTexto.toLowerCase());
  });

  if (filtrados.length === 0) {
    contenedor.innerHTML = '<p class="empty-state">No hay servidores coincidentes.</p>';
    return;
  }

  const fragmento = document.createDocumentFragment();

  filtrados.forEach(srv => {
    const id = srv.id;
    const hostname = srv.hostname || srv.nombre || `srv-${id}`;
    const estado = (srv.estado || 'OFFLINE').toUpperCase();
    const cpu = Number(srv.consumo_cpu ?? srv.cpu_uso ?? 0).toFixed(1);
    const ram = Number(srv.ram_libre_mb ?? srv.ram_libre ?? 0).toFixed(0);
    const ip = srv.ip_local || '127.0.0.1';

    const card = document.createElement('article');
    card.className = `card-server card-server--${estado.toLowerCase()}`;
    card.innerHTML = `
      <header class="card-server__header">
        <h3 class="card-server__title">${escapeHtml(hostname)}</h3>
        <span class="badge badge--${estado.toLowerCase()}">${escapeHtml(estado)}</span>
      </header>
      <p class="card-server__metric">IP: <strong>${escapeHtml(ip)}</strong></p>
      <p class="card-server__metric">CPU: <strong>${cpu}%</strong></p>
      <p class="card-server__metric">RAM libre: <strong>${ram} MB</strong></p>
      <div class="card-server__actions">
        <button class="btn btn--primary" data-action="restart" data-id="${id}">Reiniciar</button>
        <button class="btn btn--ghost" data-action="detail" data-id="${id}">Detalle</button>
      </div>
    `;
    fragmento.appendChild(card);
  });

  contenedor.appendChild(fragmento);
}

export function renderKPIs(servidores) {
  if (!kpiActivos || !kpiCpu || !kpiRam) return;
  const total = servidores.length;
  if (total === 0) {
    kpiActivos.textContent = '0 / 0';
    kpiCpu.textContent = '0.0%';
    kpiRam.textContent = '0 MB';
    return;
  }
  const activos = servidores.filter(s => (s.estado || '').toUpperCase() === 'ONLINE').length;
  const sumaCpu = servidores.reduce((acc, s) => acc + Number(s.consumo_cpu ?? s.cpu_uso ?? 0), 0);
  const sumaRam = servidores.reduce((acc, s) => acc + Number(s.ram_libre_mb ?? s.ram_libre ?? 0), 0);
  kpiActivos.textContent = `${activos} / ${total}`;
```

3.5 Patrones de Rendimiento: Event Delegation y Debouncing

Event Delegation (Burbujeo de Eventos): En vez de adjuntar 100 `addEventListener('click')` individuales a cada botón de la tabla (desperdiciando memoria RAM y creando fugas por listeners huérfanos), se coloca **un único listener en el contenedor padre**. Gracias a la fase de burbujeo (*Event Bubbling*), `e.target.closest()` intercepta el clic y obtiene los atributos `data-action`.
Debouncing: Evita saturar el navegador con filtrados innecesarios mientras el usuario escribe en el teclado. Pospospone la ejecución de la búsqueda hasta que hayan transcurrido 250 ms sin que se pulse otra tecla.

JAVASCRIPT: EVENT DELEGATION Y DEBOUNCE (js/events.js) — js/events.js

```
const contenedor = document.getElementById('contenedor-servidores');
const buscador = document.getElementById('input-buscador');

export function setupEventos({ onFiltro, onAccion }) {
  if (contenedor) {
    contenedor.addEventListener('click', (e) => {
      const btn = e.target.closest('button[data-action]');
      if (!btn) return;
      const accion = btn.dataset.action;
      const id = Number(btn.dataset.id);
      onAccion(accion, id);
    });
  }

  if (buscador) {
    let timer;
    buscador.addEventListener('input', (e) => {
      clearTimeout(timer);
      timer = setTimeout(() => {
        onFiltro(e.target.value);
      }, 250);
    });
  }
}
```

JAVASCRIPT: ORQUESTADOR PRINCIPAL (js/app.js) — js/app.js

```
import { fetchServidores, fetchMetricas } from './api.js';
import { estadoApp, actualizarEstado } from './state.js';
import { renderServidores, renderKPIs } from './render.js';
import { setupEventos } from './events.js';

async function sincronizar() {
  actualizarEstado({ cargando: true, error: null });
  try {
    const [servidores, metricas] = await Promise.all([
      fetchServidores().catch(err => estadoApp.servidores),
      fetchMetricas().catch(err => null)
    ]);

    actualizarEstado({ servidores, metricas, cargando: false });
    renderServidores(estadoApp.servidores, estadoApp.filtroTexto);
    renderKPIs(estadoApp.servidores);
  } catch (err) {
    actualizarEstado({ error: err.message, cargando: false });
  }
}

document.addEventListener('DOMContentLoaded', () => {
  setupEventos({
    onFiltro: (texto) => {
      actualizarEstado({ filtroTexto: texto });
      renderServidores(estadoApp.servidores, texto);
    },
    onAccion: (acc, id) => alert(`Acción ${acc} en servidor ID ${id}`)
  });

  sincronizar();
  setInterval(sincronizar, 5000);
});
```

Laboratorio 2.1 — Prueba de Rendimiento DOM: En el navegador, abre DevTools > Performance y escribe rápidamente en el buscador. Observa cómo el Debounce evita recalcular el render en cada pulsación, manteniendo una tasa estable de 60 FPS sin tirones visuales.

MÓDULO 3: NODE.JS, EXPRESS Y CIBERSEGURIDAD PRODUCTIVA

OWASP DEFENSIVE API

4.1 Teoría: Criptografía en Tokens JWT (RFC 7519) y Timing Attacks

Estructura y Validación de JWT: Un token consta de `Header.Payload.Signature`. El servidor no almacena sesiones en disco ni consulta la base de datos en cada request; simplemente recalcula `HMAC-SHA256(Header.Payload, Secret)`. Si la firma coincide y la fecha `exp` es válida, la petición pasa de inmediato.
Vulnerabilidad de Timing Attack en Autenticación: Si un endpoint de login busca un usuario y no lo encuentra, retorna en ~2 ms. Si el usuario existe, ejecuta `bcrypt.compare` que demora ~120 ms por diseño computacional. Un atacante midiendo el tiempo de respuesta con un script puede enumerar qué correos existen en el sistema. **Solución Defensiva:** Ante usuarios inexistentes, se ejecuta una comparación dummy contra un hash bcrypt válido precalculado (DUMMY_HASH), igualando el tiempo de respuesta a nivel milimétrico.

4.2 Configuración del Proyecto Node.js (api-node/)

JSON: ESPECIFICACIÓN DE DEPENDENCIAS (package.json) — package.json

```json
{
  "name": "api-auth",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "express-rate-limit": "^7.3.1",
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.2",
    "zod": "^3.23.8"
  }
}
```

4.3 Validación Estricta de Entorno: src/config/env.js

JAVASCRIPT: ESQUEMA ZOD PARA VARIABLES DE ENTORNO — src/config/env.js

```javascript
import dotenv from 'dotenv';
import { z } from 'zod';

dotenv.config();

const schema = z.object({
  PORT: z.coerce.number().default(3000),
  JWT_SECRET: z.string().min(32, 'JWT_SECRET requiere al menos 32 caracteres criptográficos'),
  JWT_EXPIRES_IN: z.string().default('15m'),
  REFRESH_SECRET: z.string().min(32, 'REFRESH_SECRET requiere al menos 32 caracteres'),
  REFRESH_EXPIRES_IN: z.string().default('7d'),
  CORS_ORIGIN: z.string().default('http://localhost:5173'),
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development')
});

const parsed = schema.safeParse(process.env);

if (!parsed.success) {
  console.error('❌ ERROR FATAL: Variables de entorno incorrectas:');
  console.error(JSON.stringify(parsed.error.format(), null, 2));
  process.exit(1);
}

export const env = parsed.data;
```

4.4 El Algoritmo Bcrypt: Salt Dinámica y Cost Factor

¿Por qué no usar SHA-256 para contraseñas? Funciones hash como SHA-256 o MD5 están optimizadas para hardware de alta velocidad. Una GPU moderna calcula miles de millones de hashes por segundo, permitiendo ataques de fuerza bruta con tablas arcoíris (*rainbow tables*). **Bcrypt** incorpora un factor de costo computacional (`ROUNDS = 12`) que añade retraso deliberado e inyecta una sal (*salt*) aleatoria única por usuario, imposibilitando los ataques de diccionario precomputados.

JAVASCRIPT: CRIPTOGRAFÍA DEFENSIVA (src/utils/hash.js & jwt.js) — src/utils/

```
// src/utils/hash.js
import bcrypt from 'bcryptjs';
const ROUNDS = 12;
export const hashPassword = (plain) => bcrypt.hash(plain, ROUNDS);
export const comparePassword = (plain, hash) => bcrypt.compare(plain, hash);

// src/utils/jwt.js
import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';

export function firmarAccessToken(payload) {
  return jwt.sign(payload, env.JWT_SECRET, { expiresIn: env.JWT_EXPIRES_IN, algorithm: 'HS256' });
}

export function verificarAccessToken(token) {
  return jwt.verify(token, env.JWT_SECRET, { algorithms: ['HS256'] });
}
```

4.5 Middlewares de Seguridad y Manejo Centralizado de Errores

JAVASCRIPT: MIDDLEWARES DE AUTORIZACIÓN Y RATE LIMITING — src/middlewares/

```
// src/middlewares/auth.js
import { verificarAccessToken } from '../utils/jwt.js';

export function verificarTokenJWT(req, res, next) {
  const cabecera = req.headers.authorization;
  if (!cabecera || !cabecera.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Token no proporcionado' });
  }
  try {
    req.usuario = verificarAccessToken(cabecera.slice(7));
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Token expirado o inválido' });
  }
}

// src/middlewares/rateLimit.js
import rateLimit from 'express-rate-limit';
export const limitadorLogin = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Demasiados intentos fallidos. Bloqueado por 15 minutos.' }
});

// src/middlewares/errorHandler.js
export function errorHandler(err, req, res, next) {
  console.error('[EXCEPCIÓN INTERNA]', err);
  res.status(500).json({ error: 'Error interno del servidor' });
}
```

4.6 Controlador de Autenticación con Hash Dummy

JAVASCRIPT: CONTROLADOR DE LOGIN Y REGISTRO — src/controllers/auth.controller.js

```
import { z } from 'zod';
import { hashPassword, comparePassword } from '../utils/hash.js';
import { firmarAccessToken } from '../utils/jwt.js';

const usuarios = [];
const DUMMY_HASH = '$2a$12$e8k6mN1rN8P5E1g0oE7cEuZp4iN9fS5k0G0mN1rN8P5E1g0oE7cEu';

export async function login(req, res, next) {
  try {
    const { email, password } = req.body;
    if (!email || !password) return res.status(400).json({ error: 'Email y contraseña requeridos' });

    const usuario = usuarios.find(u => u.email === email);
    const hashAComparar = usuario ? usuario.passwordHash : DUMMY_HASH;
    const passwordValida = await comparePassword(password, hashAComparar);

    if (!usuario || !passwordValida) {
      return res.status(401).json({ error: 'Credenciales inválidas' });
    }

    const accessToken = firmarAccessToken({ id: usuario.id, email: usuario.email });
    res.json({ status: 'ok', accessToken });
  } catch (err) {
    next(err);
  }
}

export async function register(req, res, next) {
  try {
    const { email, password } = req.body;
    if (usuarios.some(u => u.email === email)) {
      return res.status(409).json({ error: 'El correo ya se encuentra registrado' });
    }
    const passwordHash = await hashPassword(password);
    usuarios.push({ id: usuarios.length + 1, email, passwordHash });
    res.status(201).json({ mensaje: 'Usuario creado exitosamente' });
  } catch (err) {
    next(err);
  }
}
```

4.7 Ensamblaje y Servidor: src/app.js & src/server.js

JAVASCRIPT: MONTAJE GLOBAL CON CABECERAS HELMET — src/app.js & src/server.js

```
import express from 'express';
import helmet from 'helmet';
import cors from 'cors';
import { env } from './config/env.js';
import { limitadorLogin } from './middlewares/rateLimit.js';
import { register, login } from './controllers/auth.controller.js';

export function crearApp() {
  const app = express();
  app.use(helmet());
  app.use(cors({ origin: env.CORS_ORIGIN }));
  app.use(express.json({ limit: '10kb' }));

  app.post('/api/auth/register', register);
  app.post('/api/auth/login', limitadorLogin, login);
  return app;
}

// src/server.js
const app = crearApp();
app.listen(env.PORT, () => console.log(`✔ [api-auth] Corriendo en puerto ${env.PORT}`));
```

MÓDULO 4 (PARTE I): PHP, LARAVEL, MVC Y PERSISTENCIA

ELOQUENT ORM

5.1 Teoría: Patrón Active Record y la Vulnerabilidad de Mass Assignment

Active Record vs. Data Mapper: Eloquent implementa el patrón *Active Record*, donde cada instancia de la clase modelo representa directamente una fila en la base de datos y encapsula tanto la estructura como las operaciones SQL.
Vulnerabilidad de Asignación Masiva: Al ejecutar `Servidor::create($request->all())`, si un atacante incluye en el cuerpo JSON el parámetro `is_admin: true` o `estado: 'ROOT'`, el modelo lo persistirá directamente. Para mitigar esta falla de OWASP, Laravel exige declarar la lista blanca estricta `$fillable`. Cualquier campo no listado es descartado silenciosamente.

5.2 Inicialización del Proyecto Laravel API

BASH: COMANDOS ARTISAN DE INICIALIZACIÓN — Terminal

```
composer create-project laravel/laravel api-laravel
cd api-laravel
touch database/database.sqlite
php artisan install:api
php artisan make:model Servidor -mcr
```

5.3 Migración de Esquema y Modelo Protegido: Servidor.php

PHP: MIGRACIÓN RELACIONAL Y MODELO ELOQUENT — database/migrations & app/Models/Servidor.php

```php
<?php // database/migrations/create_servidores_table.php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
  public function up(): void {
    Schema::create('servidores', function (Blueprint $table) {
      $table->id();
      $table->string('hostname', 100)->unique();
      $table->ipAddress('ip_local');
      $table->enum('estado', ['ONLINE', 'OFFLINE', 'MANTENIMIENTO'])->default('OFFLINE');
      $table->float('consumo_cpu', 8, 2)->default(0.00);
      $table->float('ram_libre_mb', 8, 2)->default(0.00);
      $table->timestamps();
    });
  }

  public function down(): void {
    Schema::dropIfExists('servidores');
  }
};

// app/Models/Servidor.php
namespace App\Models;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Servidor extends Model {
  use HasFactory;
  protected $table = 'servidores';
  protected $fillable = [
    'hostname',
    'ip_local',
    'estado',
    'consumo_cpu',
    'ram_libre_mb'
  ];
  protected $casts = [
    'consumo_cpu' => 'float',
    'ram_libre_mb' => 'float'
  ];
}
```

5.4 Desacoplamiento de Validaciones y Transformación de Datos

FormRequests vs Validación en el Controlador: Escribir `$request->validate()` dentro del controlador viola el principio de responsabilidad única (SRP). Los *FormRequests* encapsulan la lógica de validación e intercepción antes de que el controlador sea instanciado. Si un campo falla, Laravel interrumpe el ciclo y responde automáticamente un código `422 Unprocessable Entity` con los errores estructurados en JSON.
API Resources: Crean una capa intermedia entre la base de datos y el cliente. Permiten cambiar nombres de columnas en la tabla SQLite sin alterar el contrato JSON público consumido por el frontend.

PHP: FORMREQUESTS DE VALIDACIÓN (app/Http/Requests/) — StoreServidorRequest.php

```php
<?php
namespace App\Http\Requests;
use Illuminate\Foundation\Http\FormRequest;

class StoreServidorRequest extends FormRequest {
  public function authorize(): bool { return true; }

  public function rules(): array {
    return [
      'hostname' => 'required|string|max:100|unique:servidores,hostname',
      'ip_local' => 'required|ip',
      'estado' => 'required|in:ONLINE,OFFLINE,MANTENIMIENTO',
      'consumo_cpu' => 'nullable|numeric|min:0|max:100',
      'ram_libre_mb' => 'nullable|numeric|min:0',
    ];
  }
}

// app/Http/Requests/UpdateServidorRequest.php
class UpdateServidorRequest extends FormRequest {
  public function authorize(): bool { return true; }

  public function rules(): array {
    return [
      'hostname' => 'sometimes|string|max:100',
      'ip_local' => 'sometimes|ip',
      'estado' => 'sometimes|in:ONLINE,OFFLINE,MANTENIMIENTO',
    ];
  }
}
```

PHP: CAPA DE TRANSFORMACIÓN (app/Http/Resources/ServidorResource.php) — ServidorResource.php

```php
<?php
namespace App\Http\Resources;
use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class ServidorResource extends JsonResource {
  public function toArray(Request $request): array {
    return [
      'id' => $this->id,
      'hostname' => $this->hostname,
      'ip_local' => $this->ip_local,
      'estado' => $this->estado,
      'consumo_cpu' => $this->consumo_cpu,
      'ram_libre_mb' => $this->ram_libre_mb,
      'creado_en' => $this->created_at?->toIso8601String(),
    ];
  }
}
```

5.5 Controlador RESTful y Rutas Sanctum

PHP: CONTROLADOR REST API (app/Http/Controllers/Api/ServidorController.php) — ServidorController.php

```php
<?php
namespace App\Http\Controllers\Api;
use App\Http\Controllers\Controller;
use App\Http\Requests\StoreServidorRequest;
use App\Http\Requests\UpdateServidorRequest;
use App\Http\Resources\ServidorResource;
use App\Models\Servidor;

class ServidorController extends Controller {
  public function index() {
    return ServidorResource::collection(Servidor::orderBy('hostname')->paginate(15));
  }

  public function store(StoreServidorRequest $request) {
    $servidor = Servidor::create($request->validated());
    return (new ServidorResource($servidor))->response()->setStatusCode(201);
  }

  public function update(UpdateServidorRequest $request, Servidor $servidor) {
    $servidor->update($request->validated());
    return new ServidorResource($servidor);
  }

  public function destroy(Servidor $servidor) {
    $servidor->delete();
    return response()->json(['mensaje' => 'Eliminado con éxito']);
  }
}
```

5.6 Enrutamiento con Autenticación Sanctum

PHP: ENDPOINT AUTH Y RUTAS DE RECURSO (routes/api.php) — routes/api.php

```php
<?php
use App\Http\Controllers\Api\ServidorController;
use Illuminate\Support\Facades\Route;
use Illuminate\Support\Facades\Auth;

Route::post('/login', function (\Illuminate\Http\Request $request) {
  $credenciales = $request->validate([
    'email' => 'required|email',
    'password' => 'required'
  ]);

  if (!Auth::attempt($credenciales)) {
    return response()->json(['error' => 'Credenciales inválidas'], 401);
  }

  $token = Auth::user()->createToken('api')->plainTextToken;
  return response()->json(['token' => $token]);
});

Route::middleware('auth:sanctum')->group(function () {
  Route::apiResource('servidores', ServidorController::class);
});
```

5.7 Factories y Seeders para Pruebas

PHP: GENERACIÓN DE DATOS SINTÉTICOS (database/seeders/) — ServidorSeeder.php

```php
// database/seeders/ServidorSeeder.php
Servidor::factory()->count(25)->create();
```

MÓDULO 4 (PARTE II): PYTHON ASÍNCRONO, FASTAPI Y MULTI-WORKERS

ASGI CONCURRENCY

6.1 Teoría: WSGI vs. ASGI y el Bloqueo del GIL de CPython

WSGI (Sincrónico): En frameworks tradicionales como Flask o Django antiguo, cada hilo atiende una sola conexión HTTP. Si la petición realiza una consulta lenta, ese hilo queda bloqueado sin poder recibir nuevos clientes.
ASGI (Asíncrono): FastAPI corre sobre Uvicorn utilizando el bucle de eventos `asyncio`. Las peticiones I/O se pausan con `await`, liberando el hilo para atender miles de conexiones concurrentes.
El Global Interpreter Lock (GIL): CPython no puede ejecutar bytecode de Python en paralelo sobre múltiples núcleos dentro del mismo proceso. Para saturar servidores multinúcleo se utiliza un pool de procesos trabajadores independientes: `Workers = (2 × Cores) + 1`. Cada worker posee su propio PID y procesa solicitudes en paralelo.

6.2 Schemas Pydantic v2 y Métrica psutil

PYTHON: VALIDACIÓN DE TIPOS Y LECTURA DE HARDWARE — schemas.py & telemetry.py

```python
# app/models/schemas.py
from pydantic import BaseModel, Field, IPvAnyAddress, ConfigDict

class MetricsHardware(BaseModel):
  cpu_porcentaje: float = Field(..., ge=0, le=100)
  ram_libre_mb: float = Field(..., ge=0)
  worker_pid: int = Field(..., description="PID del proceso que atendió")

# app/services/telemetry.py
import psutil, os

def obtener_metricas() -> dict:
  mem = psutil.virtual_memory()
  return {
    "cpu_porcentaje": psutil.cpu_percent(interval=None),
    "ram_libre_mb": round(mem.available / (1024 * 1024), 2),
    "worker_pid": os.getpid(),
  }
```

6.3 Router de Sistema y Servidores

PYTHON: ENDPOINTS FASTAPI DE TELEMETRÍA — app/routers/system.py

```python
from fastapi import APIRouter
from ..models.schemas import MetricsHardware
from ..services.telemetry import obtener_metricas

router = APIRouter(prefix="/api/v1/system", tags=["system"])

@router.get("/metrics", response_model=MetricsHardware)
async def metrics():
  return obtener_metricas()
```

6.4 El Main Thread en GUIs y Concurrencia con Tkinter

¿Por qué se congela una aplicación de escritorio? El bucle de eventos gráficos (`root.mainloop()`) corre estrictamente en el hilo principal (*Main Thread*). Si ejecutas una llamada de red de 3 segundos dentro de un botón ordinario, el sistema operativo deja de recibir eventos de ventana y muestra el aviso "La aplicación no responde".
Solución Thread-Safe: La tarea pesada se delega a un hilo de trabajo en segundo plano (`threading.Thread(daemon=True)`). Dado que las librerías gráficas no admiten mutaciones visuales desde hilos secundarios, el resultado se despacha al hilo principal mediante `root.after(0, callback)`.

PYTHON: SERVIDOR FASTAPI Y ARRANQUE MULTI-WORKER — app/main.py & backend/run.sh

```python
# app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from .services.telemetry import obtener_metricas

app = FastAPI(title="Onetel Telemetry Core")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"])

@app.get("/api/v1/system/metrics")
async def get_metrics():
  return obtener_metricas()

# backend/run.sh
uvicorn app.main:app --host 127.0.0.1 --port 8001 --workers 4
```

PYTHON: GUI THREAD-SAFE CON PROGRESO SEGURO — desktop/app_desktop.py

```python
import tkinter as tk
from tkinter import ttk, messagebox
import threading, requests, time

class MonitorApp:
  def __init__(self, root):
    self.root = root
    self.root.title("Onetel Local Monitor")
    self.lbl = ttk.Label(root, text="En espera...")
    self.lbl.pack(pady=10)
    ttk.Button(root, text="Consultar Telemetría", command=self.consultar).pack(pady=5)

  def consultar(self):
    threading.Thread(target=self._hilo_segundo_plano, daemon=True).start()

  def _hilo_segundo_plano(self):
    try:
      res = requests.get("http://127.0.0.1:8001/api/v1/system/metrics", timeout=4).json()
      txt = f"Worker PID: {res['worker_pid']} | CPU: {res['cpu_porcentaje']}%"
      self.root.after(0, lambda: self.lbl.config(text=txt))
    except Exception as e:
      self.root.after(0, lambda: messagebox.showerror("Fallo", str(e)))

if __name__ == "__main__":
  root = tk.Tk()
  app = MonitorApp(root)
  root.mainloop()
```

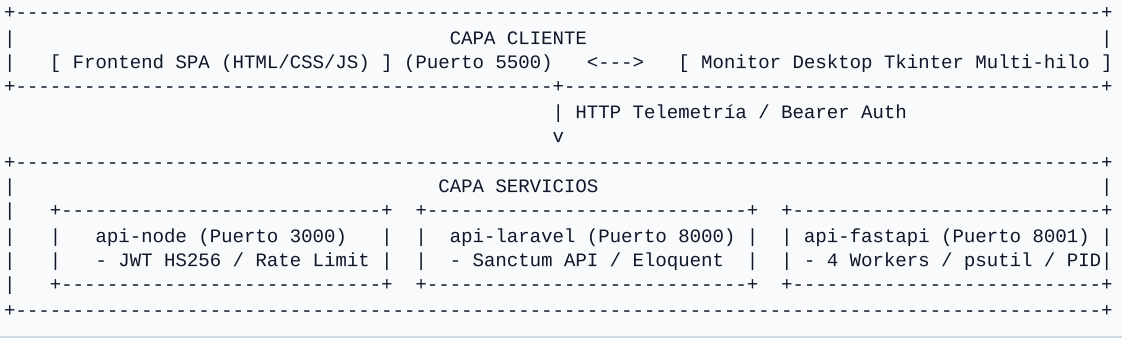
MÓDULO 5: ARQUITECTURA MONOREPO Y ORQUESTACIÓN DISTRIBUIDA

ECOSISTEMA INTEGRADO

7.1 Filosofía Monorepo y Coordinación de Procesos

¿Por qué Monorepo en Sistemas Distribuidos? Un monorepo reúne las capas de frontend, backend y microservicios en un único árbol de control de versiones. Esto asegura que los cambios en contratos de APIs (schemas) se prueben atómicamente contra la UI en un solo commit. El script `dev-all.sh` orquesta los 3 servidores en segundo plano y captura la señal de salida mediante `trap EXIT` para apagar limpiamente todos los subprocesos de Node, Laravel y FastAPI.

DIAGRAMA: ECOSISTEMA MULTI-STACK INTEGRADO — Topología Local



7.2 Estructura del Monorepo Distribuido

TEXT: ÁRBOL DE DIRECTORIOS MONOREPO — onetel-fullstack/

```
onetel-fullstack/
├── .gitignore / .env.example / README.md
├── scripts/     (verify-env.sh, setup-server.sh, dev-all.sh)
├── frontend/    (index.html, css/, js/)
├── api-node/    (Node.js 20/24 + Express 5 Auth)
├── api-laravel/ (Laravel 11/12 + Sanctum CRUD)
├── api-fastapi/ (FastAPI + 4 Workers Telemetry)
└── desktop/     (Tkinter Thread-Safe Monitor)
```

7.3 Orquestador Maestro de Servicios

BASH: ORQUESTADOR MAESTRO (scripts/dev-all.sh) — scripts/dev-all.sh

```bash
#!/bin/bash
set -e
echo "=== Arrancando ecosistema distribuido Onetel ==="

(cd api-node && npm run dev) &
NODE_PID=$!

(cd api-laravel && php artisan serve --port=8000) &
LARAVEL_PID=$!

(cd api-fastapi && source .venv/bin/activate && \
 uvicorn app.main:app --host 127.0.0.1 --port 8001 --workers 4) &
FASTAPI_PID=$!

echo "✔ Servicios corriendo: Node PID=$NODE_PID, Laravel PID=$LARAVEL_PID, FastAPI PID=$FASTAPI_PID"
echo "Presiona Ctrl+C para detener todos los servidores simultáneamente."

trap "echo 'Apagando procesos...'; kill $NODE_PID $LARAVEL_PID $FASTAPI_PID 2>/dev/null" EXIT
wait
```

7.4 Checklist de Auditoría Técnica (20 Criterios Mandatorios)

#	Criterio de Aceptación	Método de Comprobación y Resultado Esperado
01	Entorno reproducible	`./scripts/verify-env.sh` retorna código 0.
02	Clave SSH Ed25519 activa	`ssh -T git@github.com` confirma autenticación válida.
03	Hook pre-commit operativo	Commit con archivo `.env` es abortado con exit 1.
04	Hook post-receive funcional	`git push local-deploy main` actualiza `/var/www/app`.
05	UI Responsive sin overflow	Inspección en DevTools (320px a 1920px) sin scroll horizontal.
06	Accesibilidad Lighthouse	Puntuación de accesibilidad ≥ 95 en auditoría Lighthouse.
07	Sanitización dinámica XSS	Inyección de `<img onerror>` se renderiza como texto literal.
08	Manipulación DOM eficiente	`DocumentFragment` utilizado en `renderServidores()`.
09	Rate Limiting perimetral	5 peticiones erróneas consecutivas activan código HTTP 429.
10	Criptografía defensiva	Algoritmo `bcrypt` configurado con `ROUNDS = 12`.
11	Timing Attack mitigado	Comparación dummy con DUMMY_HASH ante usuario inexistente.
12	Protección con Helmet	Cabeceras HTTP de seguridad verificadas con `curl -I`.
13	Migraciones reproducibles	`php artisan migrate:fresh --seed` ejecuta limpiamente.
14	Validación 422 estructurada	POST con cuerpo inválido retorna código 422 con detalles.
15	Anti Mass Assignment	Propiedad `$fillable` estricta en modelo Eloquent.
16	Multi-Worker FastAPI	4 workers activos + 1 master en `ps -ef | grep uvicorn`.
17	Balanceo de carga	Llamadas a `/metrics` alternan los `worker_pid`.
18	Concurrencia ApacheBench	`ab -n 200 -c 20` completa con 0 fallas de conexión.
19	GUI Desktop Thread-Safe	La ventana se arrastra fluidamente mientras la barra carga.
20	Higiene del repositorio	`git log -p` completamente libre de secretos o credenciales.

7.5 Batería de Pruebas de Estrés Final

BASH: VALIDACIÓN DE CARGA Y SEGURIDAD CONSOLIDADAS — stress-test.sh

```bash
# 1. Prueba de Rate Limiting (Esperado: 401 401 401 401 401 429 429...)
for i in {1..8}; do
  curl -s -o /dev/null -w "%{http_code} " -X POST http://localhost:3000/api/auth/login \
    -H "Content-Type: application/json" -d '{"email":"test@onetel.cloud","password":"wrong"}'
done
echo ""

# 2. Prueba de balanceo de carga entre Workers FastAPI
for i in {1..4}; do
  curl -s http://127.0.0.1:8001/api/v1/system/metrics | jq -r .worker_pid
done

# 3. Test de Concurrencia con ApacheBench (200 peticiones en 20 hilos concurrentes)
ab -n 200 -c 20 http://127.0.0.1:8001/api/v1/system/metrics
```

ANEXOS: CONVENCIONES, CONFIGURACIÓN Y DEFENSA TÉCNICA

NORMAS PROFESIONALES

A. Convención de Commits Semánticos (Conventional Commits)

Tipo	Propósito Arquitectónico	Ejemplo Estandarizado
`feat`	Nueva funcionalidad agregada al sistema	`feat(auth): implementar rotacion tokens`
`fix`	Corrección de un error o vulnerabilidad	`fix(render): mapear atributos correctos`
`security`	Endurecimiento y defensas perimetrales	`security(auth): mitigar timing attack`
`perf`	Optimización de rendimiento y memoria	`perf(render): agrupar con DocumentFragment`

B. Plantilla Centralizada .env.example

ENV: ESPECIFICACIÓN DE VARIABLES DE ENTORNO PÚBLICAS — .env.example

```
# ========================================
# API NODE.JS (Autenticación y Seguridad)
# ========================================
PORT=3000
NODE_ENV=development
JWT_SECRET=super_secret_key_de_al_menos_32_caracteres_hex_1234
JWT_EXPIRES_IN=15m
REFRESH_SECRET=super_refresh_secret_de_al_menos_32_caracteres_5678
REFRESH_EXPIRES_IN=7d
CORS_ORIGIN=http://localhost:5500

# ========================================
# API LARAVEL (Servidores y Persistencia)
# ========================================
APP_NAME="Onetel Infrastructure"
APP_ENV=local
APP_KEY=
APP_DEBUG=true
APP_URL=http://localhost:8000

DB_CONNECTION=sqlite
DB_DATABASE=/var/www/app/database/database.sqlite

# ========================================
# API FASTAPI (Telemetría de Hardware)
# ========================================
FASTAPI_HOST=127.0.0.1
FASTAPI_PORT=8001
FASTAPI_WORKERS=4
```

C. Guía de Preguntas para Sustentación ante Comité Evaluador

1. Arquitectura Git: ¿Por qué modificar un commit histórico altera en cascada los hashes de todos sus descendientes en el DAG? (Respuesta: Por el encadenamiento criptográfico recursivo del hash padre).
2. Seguridad: ¿Por qué comparar `password === user.password` en texto plano es una negligencia crítica según OWASP? (Respuesta: Vulnerable a filtración directa de la base de datos y ataques de canal lateral).
3. Frontend: ¿Qué diferencia fundamental existe entre un Repaint y un Reflow en el Render Tree? (Respuesta: El Reflow recalcula dimensiones geométricas y posiciones; el Repaint solo redibuja píxeles de color sin alterar el layout).
4. Backend Concurrente: ¿Por qué levantar 4 workers de Uvicorn en un servidor de 4 cores físicos multiplica el rendimiento real? (Respuesta: Porque cada proceso sortea el Global Interpreter Lock de Python y atiende peticiones en paralelo).